

Indian Institute of Technology Roorkee
Department of Computer Science & Engineering
Subject: Computer Networks (CSC-303) | Project Report

PHOTON

Live Photo Sync for Groups over Unreliable Mobile Networks

A verified, resumable and ephemeral photo-sharing system built on HTTP/3, WebSocket and Redis Streams

Group No. 13

Submitted To: Prof. Sandeep Kumar

Submission Date: 3 November, 2026

Group Members & Contributions

Name	Enroll. No.	Email	Individual Contribution
Akshat Srivastava	24114010	akshat_s@cs.iitr.ac.in	System design; Redis Functions library (sessions, announce, fencing, quota); orchestrator service
Shubham Singla	24114093	shubham_s@cs.iitr.ac.in	Image server: resumable chunked upload with CRC32C, SHA-256 verification, sandboxed decoder, outbox
Yash Jain	24114108	yash_j2@cs.iitr.ac.in	Network edge (Caddy, TLS 1.3, HTTP/3), WebSocket event plane, Redis Streams catch-up and resync
Pushkar Jain	24114071	pushkar_j@cs.iitr.ac.in	Auth service: OAuth 2.0 with PKCE (Google, GitHub), EdDSA JWTs, refresh rotation; Redis ACLs
Aditya Yadav	24114007	aditya_y@cs.iitr.ac.in	Android app (Kotlin): UI, MediaStore observer, foreground service, WorkManager upload, receive pipeline
Divyanshu Meena	24114034	divyanshu_m@cs.iitr.ac.in	iOS PhotonKit, push (FCM/APNs), Docker deployment, tests and benchmarks, project report and slides

Contents

1 Introduction **2**
1.1 Background . . . 2
1.2 Motivation . . . 2
1.3 Application Domain . . . 3
1.4 Problem Statement . . . 3
1.5 Objectives and Scope . . . 3

2 Networking Background and Design Rationale **3**
2.1 Networking Concepts Used in PHOTON . . . 3
2.2 Choosing a Protocol for Each Plane . . . 4
2.3 Protocol Stack . . . 5
2.4 Reliability Concepts . . . 5

3 Proposed System **6**
3.1 Overview . . . 6
3.2 System Architecture Block Diagram . . . 6
3.3 Steps of the Proposed System . . . 7
3.4 Life of a Photo . . . 7
3.5 Wire Protocol and API . . . 8
3.6 Data Model . . . 10
3.7 State Machines . . . 10
3.8 Algorithms . . . 11
3.9 Novelty and Innovation . . . 14

4 Implementation **16**
4.1 Technology Stack . . . 16
4.2 File Inventory . . . 17
4.3 Server Implementation . . . 17
4.4 Network Edge and Deployment . . . 18
4.5 Authentication and Security . . . 18
4.6 Android Client . . . 19
4.7 iOS Client . . . 20
4.8 Testing . . . 20
4.9 Performance Evaluation . . . 21
4.10 Problems Found by Measurement . . . 23
4.11 Screenshots of the System . . . 24
4.12 Installation Guide . . . 25
4.13 Comparison with Existing Systems . . . 26

5 Conclusion **27**
5.1 Summary of Achievements . . . 27
5.2 Limitations . . . 28
5.3 Future Work . . . 28
5.4 Final Remarks . . . 28

6 Bibliography **28**

1. Introduction

1.1 Background

A group of people at a wedding, a field trip or a hackathon takes hundreds of photos, and every photo ends up on the one phone that took it. Getting the photos to everyone else is still a manual job: someone creates a shared album days later, people forward pictures one by one in a chat, or the group gives up. The photos exist; the network between the phones is what is missing.

PHOTON closes that gap. Members join a live *session* with a short code or a QR code, and from then on every photo any member takes appears in every other member's gallery, without anyone pressing a button. The server keeps a photo only for 30 minutes, long enough for the group's phones to fetch it.

Behind this simple behaviour sit most of the topics of a computer networks course:

- **Application-layer protocols.** The system uses HTTP/1.1, HTTP/2 and HTTP/3 for requests and bulk transfer, WebSocket for a long-lived event channel, and push services (FCM, APNs) as wake-up signals.
- **Transport protocols.** TCP carries the control and event planes; QUIC over UDP carries photo data, because it survives a phone moving from Wi-Fi to cellular and it does not stall every stream when one packet is lost.
- **Reliable delivery over an unreliable network.** Large uploads are cut into checksummed chunks, sent several at a time, and only the missing chunks are resent, which is selective repeat at the application layer. Every request is idempotent, so a retry after a timeout is always safe.
- **Publish/subscribe with a durable log.** Events go into a Redis Stream and are read by one loop per client, so a client that reconnects receives exactly what it missed.
- **Network security.** TLS 1.3 protects every hop. Users sign in with OAuth 2.0 and PKCE and receive signed JSON Web Tokens. Photos are content-addressed by SHA-256, and the server recomputes the hash itself instead of trusting the client.
- **Distributed state.** Sessions and photo status live in Redis and change only through atomic server-side scripts. Late messages from a slow client are rejected with *fencing tokens*, so they can never undo newer state.

1.2 Motivation

Four concrete problems drove the design of PHOTON.

1. **Sharing is manual and late.** Shared albums and chat groups need someone to pick photos and send them. By the time that happens the event is over, and many photos are never shared.
2. **Cloud albums keep everything, forever.** Existing services store every shared photo indefinitely, tie the group to one vendor's ecosystem, or recompress the images. A group that only wants to exchange photos during an event should not leave a permanent copy on someone else's server.
3. **Mobile networks are hostile to large uploads.** A 4 MB photo uploaded over a single HTTP request restarts from zero when the phone changes network, a lift cuts the signal or the operating system suspends the app. Retries that are not idempotent create duplicates, and notifications that are lost leave members with gaps.
4. **A client's claim is not proof.** If the server trusts the hash a client announces, one member can upload different bytes under the hash of someone else's real photo, and deduplication then hides the real photo from everyone. The server has to verify the bytes before telling anyone the photo exists.

1.3 Application Domain

Where PHOTON Is Used

1. Events and trips: weddings, college fests, family trips and sports days, where many people photograph the same moments and everyone wants everyone's photos.

2. Education and teams: field labs, site visits and hackathons, where a team documents its work on several phones and needs one shared record during the activity.

3. Inspections and incident reporting: surveyors, insurance assessors or volunteers who must get photos from the field to their colleagues at once, without keeping them on a third-party cloud afterwards.

4. Any short-lived group: a session lasts hours, not years, and server copies are deleted 30 minutes after they are stored, so nothing outlives the event on the server.

1.4 Problem Statement

Core Problem Statement

How can a group of phones, connected through unreliable and changing mobile networks, share every photo taken during a live session with every other member automatically, so that each photo is uploaded once, verified before anyone is told about it, delivered to every member without gaps or duplicates even across disconnections, and removed from the server within a bounded time?

1.5 Objectives and Scope

The objectives of the project were to:

1. detect a new camera photo on the device and sync it while the user is in a session, also while the app is in the background;
2. upload each photo once per session, using its SHA-256 hash as the content address;
3. show a thumbnail first, then a preview, then save the original, according to network conditions;
4. announce a photo to the group only after the server has verified its hash and stored it;
5. deliver every event to every member at least once and in order, with deduplication on the client so that each photo is processed exactly once;
6. make a photo unavailable exactly 30 minutes after it is stored and delete the file within 5 minutes after that;
7. protect every hop with TLS 1.3 and every request with a verified identity.

Out of scope for this version are video and Live Photos, photos taken before joining, long-term backup, more than one active session per user, and a web client (a browser cannot watch the phone's gallery).

2. Networking Background and Design Rationale

This chapter introduces the networking concepts PHOTON is built on and explains, for each kind of traffic, why one protocol was chosen over the alternatives.

2.1 Networking Concepts Used in PHOTON

Table 1 maps each concept to the standard that defines it and to the place where PHOTON uses it.

Concept	Standard	Role in PHOTON
HTTP semantics, status codes, idempotent methods	RFC 9110	Every REST call; the error contract maps each failure to a status code and a retry rule; chunk `PUT`s are idempotent
HTTP/2 multiplexing	RFC 9113	Several chunk uploads share one TCP connection; fallback when UDP is blocked
QUIC and HTTP/3	RFC 9000, 9114	Data plane over UDP 443: no head-of-line blocking between streams, 1-RTT setup, connection migration across networks
Alternative services (Alt-Svc)	RFC 7838	The edge advertises `h3` on UDP 443; clients switch to it and fall back to HTTP/2 automatically
TLS 1.3	RFC 8446	Every hop, including service-to-service calls and Redis
WebSocket	RFC 6455	Full-duplex event channel; binary protobuf frames; application close codes 4001–4410
OAuth 2.0 with PKCE	RFC 6749, 7636	Sign-in through Google or GitHub without a client secret on the phone
JSON Web Token, EdDSA	RFC 7519, 8037	Access tokens and per-photo upload tokens, verified locally by each service
CRC32C, SHA-256	RFC 3720, FIPS 180-4	Per-chunk integrity check; content address and end-to-end verification of each photo
Reverse proxy, forwarded client address	RFC 7239	Caddy terminates TLS; `X-Forwarded-For` is trusted only from known proxies, for per-IP rate limits
Publish/subscribe over a durable log	Redis Streams	Gap-free event delivery with a resumable cursor
Push notifications	FCM, APNs	Best-effort wake-up hint for phones without an open socket
Port forwarding	Android `adb reverse`	A phone on USB reaches the laptop's development server through a tunnel

Table 1: Networking concepts, their standards, and their role in PHOTON

2.2 Choosing a Protocol for Each Plane

PHOTON separates its traffic into four planes, because each has different needs: small requests that must be answered, events that must reach the phone as soon as they happen, megabytes of photo data that must survive bad networks, and calls between services that must be authenticated. Table 2 lists the options considered for each plane.

Plane	Options considered	Chosen	Reason
Control (client to orchestrator)	Raw TCP protocol, gRPC, HTTP with JSON, HTTP with protobuf	HTTPS with protobuf bodies (`application/x-protobuf`)	FastAPI serves HTTP, not raw sockets; status codes, headers, proxies and retries come for free; protobuf keeps bodies small
Events (orchestrator to client)	Polling, long polling, Server-Sent Events, push only, WebSocket	WebSocket, plus FCM/APNs as a hint	One open socket delivers an event within a few milliseconds; push alone may be dropped, delayed or merged, so it only wakes the app
Data (client and image server)	One large HTTP/1.1 `PUT`, raw TCP, HTTP/2, HTTP/3	HTTP/3 over QUIC, HTTP/2 fallback, chunked and resumable	QUIC survives network changes and avoids head-of-line blocking; chunks bound the cost of a failure to 1 MiB
Internal (service to service)	Shared database writes, message queue, HTTPS	HTTPS with a service JWT (mTLS optional)	Only the image server may report a photo as stored; every report is authenticated and fenced

Table 2: Protocol options considered for each plane

Why not a hand-built TCP protocol? An early version of the design used raw TCP between the phone and the server and a hand-written fallback from QUIC to TCP. Both were dropped. HTTP already defines request framing, status codes, authentication headers and caching rules, and every proxy, load balancer and monitoring tool understands it. Standard HTTP/3 clients (Cronet on Android, `URLSession` on iOS) already fall back to HTTP/2 when UDP is blocked. A custom protocol would have repeated all of this with more bugs.

Why QUIC for photo data? Over TCP, one lost segment stalls every HTTP/2 stream on the connection until it is retransmitted, so a lost packet in chunk 2 also delays chunks 3 and 4. QUIC delivers each stream independently. QUIC also identifies a connection by a connection ID rather than by the address and port pair, so when a phone moves from Wi-Fi to LTE its new address can continue the same connection, where TCP would have to start again with a new handshake.

2.3 Protocol Stack

Figure 1 shows the protocol stack of each plane. All planes share the same payload format (protobuf) and the same network layer; they differ in the application and transport protocols.

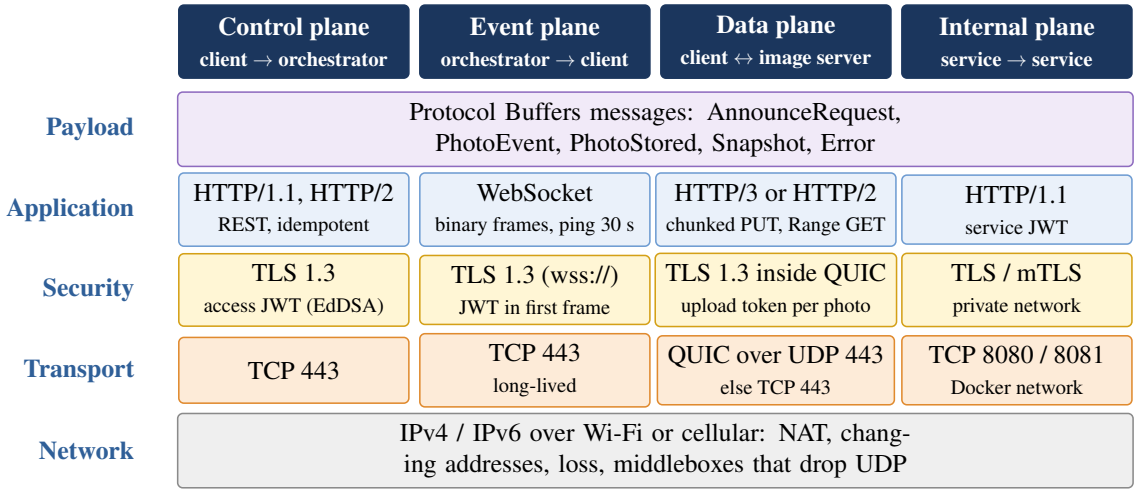


Figure 1: Protocol stack of the four planes

2.4 Reliability Concepts

Four ideas from networking and distributed systems recur throughout the design.

1. **At-least-once delivery plus idempotency gives an exactly-once effect.** The network may duplicate or drop messages, so the sender retries and the receiver makes a repeat harmless. PHOTON's chunk uploads, `complete` calls, reports and event deliveries are all built this way.
2. **Selective repeat.** As in the sliding-window protocols of the transport layer, the receiver tells the sender exactly which pieces arrived, and only the missing ones are sent again (Section 3.8.3).
3. **Leases.** A reservation that is not renewed expires on its own, so a client that disappears cannot hold a resource forever. Each upload holds a 15-minute lease that every chunk renews.
4. **Fencing tokens.** Every message that changes upload state names the `upload_id` it belongs to, and the server applies it only if that id is still the live one, so a delayed message from an old attempt is rejected.

Retries use exponential backoff with jitter, so that phones which lost the network together do not all

retry at the same instant. The delay before attempt k is

$$d_k = \tfrac{1}{2}s_k + U\left(0,\ \tfrac{1}{2}s_k\right), \qquad s_k = \min\left(60\text{ s},\ 1\text{ s}\cdot 2^k\right), \tag{1}$$

where $U(a,b)$ is a uniformly random value. A `Retry-After` from the server, when present, replaces d_k.

3. Proposed System

3.1 Overview

PHOTON is a client–server system with three server services, three data stores and native mobile clients. One rule shapes the whole design:

Design Rule

A photo is announced to the session only after the image server has recomputed its SHA-256 over the received bytes, found it equal to the hash the uploader claimed, and stored the file. Until then the photo does not exist for anyone else.

Each kind of state has exactly one writer. The **orchestrator** owns sessions, members, photo status and event fan-out, and never touches image bytes. The **image server** owns the bytes, thumbnails, previews and deletion, and never changes session state or talks to members; it only reports to the orchestrator. The **Auth** service owns the signing key and issues tokens, but is not in the path of every request, because the other services verify tokens locally with its public key. The **client** owns the gallery observer, the upload queue and its local record of received photos, but never decides on its own what counts as a duplicate.

3.2 System Architecture Block Diagram

Figure 2 shows the components and the protocols between them. All client traffic enters through one edge proxy, which terminates TLS 1.3 and offers HTTP/3 on UDP port 443 next to HTTP/2 on TCP port 443. The highlighted arrow is the only path by which a photo becomes visible: the image server's fenced report to the orchestrator.

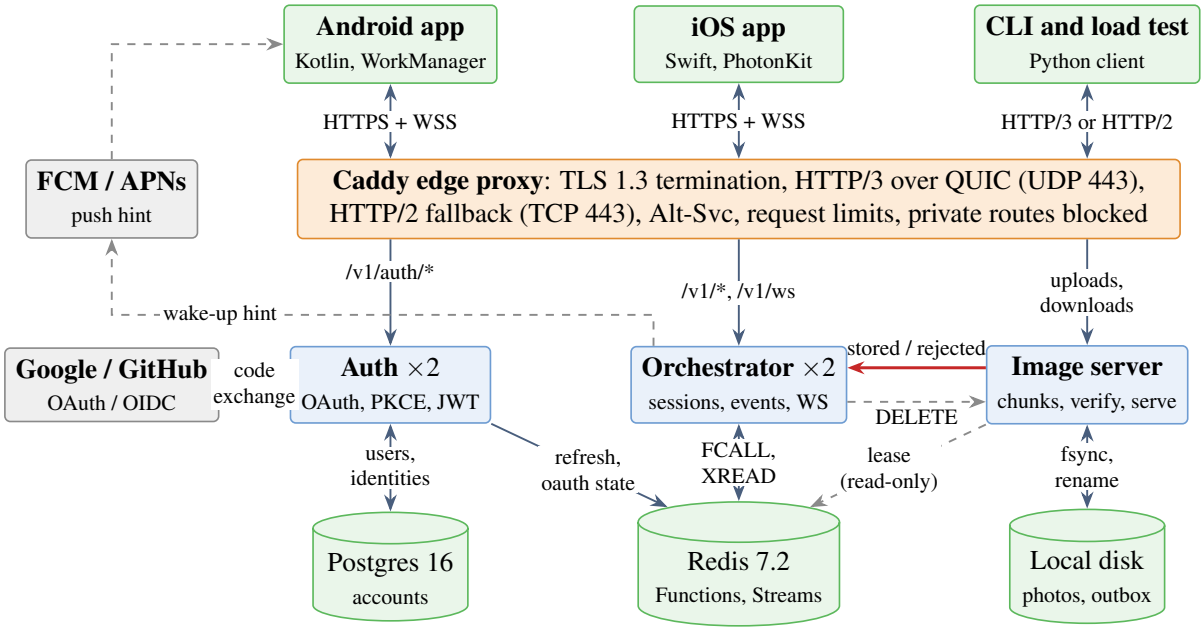


Figure 2: System architecture block diagram

3.3 Steps of the Proposed System

Step 1: Sign in. The app signs in with Google (native ID token) or GitHub (browser, Authorization Code with PKCE) and receives a 15-minute access JWT and a rotating refresh token.

Step 2: Create or join. The host creates a session and gets an 8-character code, an invite link and a QR code. Members join with any of them. One Lua script checks that the session is `ACTIVE`, not full, and that the user has no other live session.

Step 3: Connect. The app opens the WebSocket `/v1/ws`, authenticates in the first frame, loads the snapshot and resumes the event stream from its cursor. It starts a foreground service that watches the gallery.

Step 4: Detect. When the camera saves a photo, the observer records it in a local queue. A WorkManager job computes its SHA-256 and removes GPS data from a copy.

Step 5: Announce. The app sends the hash and size. The orchestrator answers `UPLOAD` (with an upload token), `ALREADY_UPLOADED` or `IN_PROGRESS`.

Step 6: Upload. The app sends 1 MiB chunks, three at a time, each with a CRC32C, then calls `complete`. After a failure it asks which chunks arrived and sends only the rest.

Step 7: Verify. The image server assembles the file, recomputes SHA-256, validates and decodes it in a sandboxed process, makes a thumbnail, and writes the metadata with `expires_at = stored_at + 30 min`.

Step 8: Report and announce. The image server reports `stored` with the `upload_id`. If the id is still the live one, the orchestrator marks the photo `STORED` and appends `PHOTO_STORED` to the session's stream in the same script.

Step 9: Deliver. Every member's WebSocket loop reads the new event and sends it; members without a socket get a push hint and catch up when they wake.

Step 10: Receive. Each member fetches the thumbnail, then the preview, then the original, each with its own JWT, and saves only the original to the gallery.

Step 11: Expire, recall or end. At `expires_at` every download is refused and the file is deleted within 5 minutes. The uploader or host can recall a photo earlier, and the host can end the session, which deletes everything.

3.4 Life of a Photo

Figure 3 follows one photo through the system as a sequence diagram. Messages 1–3 are the control plane, 4–7 the data plane, 8–10 the internal plane, and 12–14 the event and data planes on the receiving side.

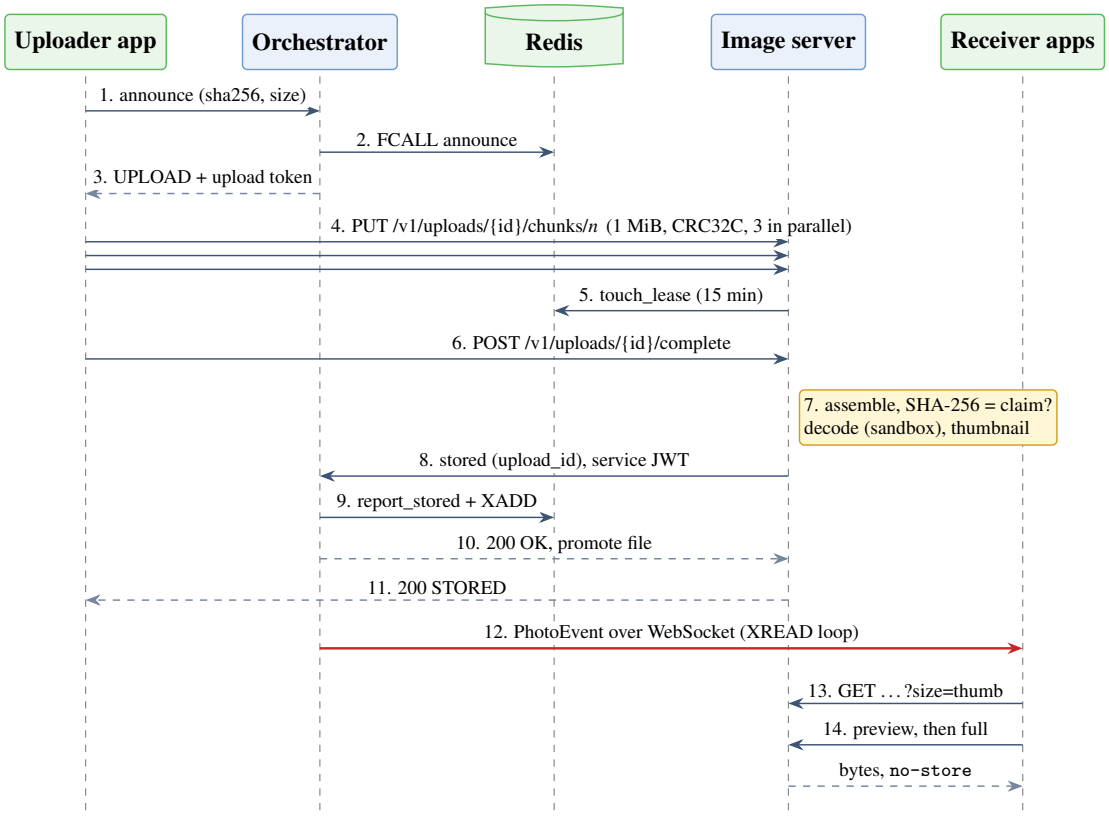


Figure 3: Life of a photo: from announce to every member's gallery

3.5 Wire Protocol and API

3.5.1 Endpoints

Endpoint	Caller	Purpose
`POST /v1/auth/oauth/start, /callback`	client	Browser sign-in with `state` and PKCE
`POST /v1/sessions, /v1/sessions/join`	client	Create (code, invite link, QR), join by code or invite
`GET /v1/sessions/{sid}/snapshot`	client	Members, live photos, recalled hashes and a cursor
`WS /v1/ws`	client	Live events; first frame is `Hello`
`POST /v1/photos/announce`	client	`UPLOAD`, `ALREADY_UPLOADED` or `IN_PROGRESS`
`PUT /v1/uploads/{id}/chunks/{n}`	client	One chunk, with `X-Chunk-CRC32C`
`GET /v1/uploads/{id}`	client	Which chunks arrived, so an upload can resume
`POST /v1/uploads/{id}/complete`	client	Idempotent: 202 while verifying, then always the same result
`GET /v1/sessions/{sid}/photos/ {hash}?size=`	client	`thumb`, `preview` or `full`; checked on every request
`POST /internal/v1/photos/stored, /rejected`	image server	Fenced report carrying the `upload_id`
`DELETE /internal/v1/photos/{sid}/ {hash}`	orchestrator	Delete after expiry, recall or end; retried until acknowledged

Table 3: Main endpoints (all bodies are protobuf)

3.5.2 Messages

All messages are defined once in `proto/photon.proto` and shared by the Python services, the Kotlin app and the Swift library. Field numbers are never reused, so old and new clients can talk to the same server. The core messages are:

```
message AnnounceRequest {
  bytes sha256 = 1;  uint64 size = 2;  string mime = 3;
  int64 taken_at_ms = 4;  string session_id = 5;
}
message AnnounceResponse {
  enum Result { UPLOAD = 0; ALREADY_UPLOADED = 1; IN_PROGRESS = 2; }
  Result result = 1;  string upload_id = 2;  string upload_url = 3;
  string upload_token = 4;  uint32 chunk_size_hint = 5;
}
message PhotoEvent {
  enum Type { PHOTO_STORED = 0; PHOTO_RECALLED = 1; MEMBER_JOINED = 2;
              MEMBER_LEFT = 3; SESSION_ENDED = 4; HOST_CHANGED = 5; }
  string event_id = 1;  Type type = 2;  string session_id = 3;  bytes sha256 = 4;
  string uploader_id = 5;  uint64 size = 6;  int64 taken_at_ms = 7;
  int64 expires_at_ms = 8;  string member_id = 9;
}
message Error {
  string code = 1;  string message = 2;  bool retryable = 3;
  uint32 retry_after_s = 4;  repeated uint32 missing_chunks = 5;
}
```

A `PhotoEvent` carries no download link. Receivers fetch by hash with their own JWT, so a leaked event is useless on its own.

3.5.3 Error Contract

Every error has one status code, one machine-readable code and one client action (Table 4). The client retries only when `retryable` is true and waits `retry_after_s` when it is set.

Status	Codes	Client action
400	`INVALID_REQUEST` (lists missing chunks)	Fix the request; send the missing chunks
401	`TOKEN_EXPIRED`, `UNAUTHENTICATED`	Refresh once, then retry
403	`NOT_A_MEMBER`, `FORBIDDEN`	Stop; rejoin if the session still exists
404	`UNKNOWN_UPLOAD`, `PHOTO_NOT_FOUND`, `SESSION_NOT_FOUND`	Drop the task; non-members get 404 so existence is not leaked
409	`ALREADY_IN_SESSION`, `UPLOAD_SUPERSEDED`, `UPLOAD_FINALIZED`	Stop this upload
410	`SESSION_ENDED`, `PHOTO_EXPIRED`, `PHOTO_RECALLED`, `RESYNC_REQUIRED`	Final: clear local state, or load the snapshot
413 / 415	`TOO_LARGE`, `UNSUPPORTED_FORMAT`	Stop; do not retry
422	`HASH_MISMATCH`, `INVALID_IMAGE`	Stop; counts as a strike (3 strikes block uploads)
429	`RATE_LIMITED`, `TOO_MANY_UPLOADS`, `QUOTA_EXCEEDED`	Retry after `Retry-After`
503	`BUSY`, `PROCESSING_FAILED`	Retry with backoff and jitter (Equation 1)

Table 4: Error contract

3.5.4 WebSocket Protocol

The WebSocket carries binary protobuf frames. The client's first frame must be `Hello` with its access token, session id and last event id; the token is never put in the URL, where proxies would log it. The server answers with `ready_cursor`, then sends `PhotoEvent` frames, and answers each `ping` with a `pong`. The server closes the socket with an application close code that tells the client what to do next (Table 5).

Code	Meaning	Client action
4001	`TOKEN_EXPIRED` (sent when the JWT expires)	Refresh the token, reconnect from the last event id
4400	`INVALID_REQUEST` (no `Hello` in time)	Fix the client; reconnect
4401	`UNAUTHENTICATED`	Sign in again
4403	`NOT_A_MEMBER`	Leave the session screen; rejoin if allowed
4409	`RESYNC_REQUIRED`	Load the snapshot, continue from its cursor
4410	`SESSION_ENDED`	Stop syncing
1011	Server error	Reconnect with backoff

Table 5: WebSocket close codes

3.6 Data Model

Session state is short-lived by design and lives in Redis; accounts must survive a restart and live in Postgres (Figure 4). Every key of one session contains the hash tag `{sid}`, so all of them fall into the same Redis Cluster slot and one script can update them atomically. The event stream's ids are `<milliseconds>-<sequence>` from the shared clock, so they increase within a session and double as the client's cursor.

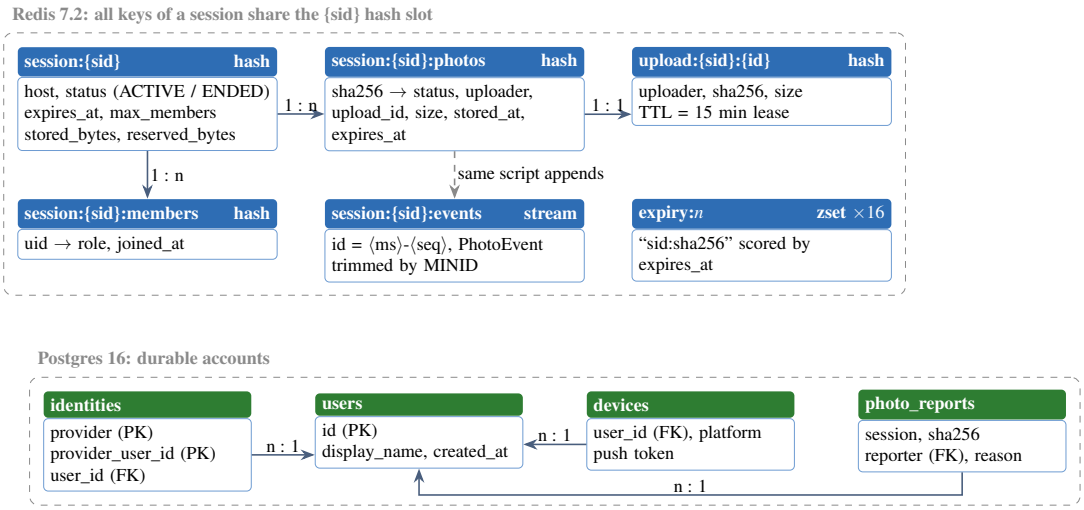


Figure 4: Data model: Redis keys for session state and Postgres tables for accounts

3.7 State Machines

A photo's status in the session moves through the states of Figure 5. Every transition is one Lua function that also checks that the session is `ACTIVE`, so nothing can succeed after the session ends. `EXPIRED` and `RECALLED` stay as tombstones until the session ends, so a late announce of the same photo does not upload it again.

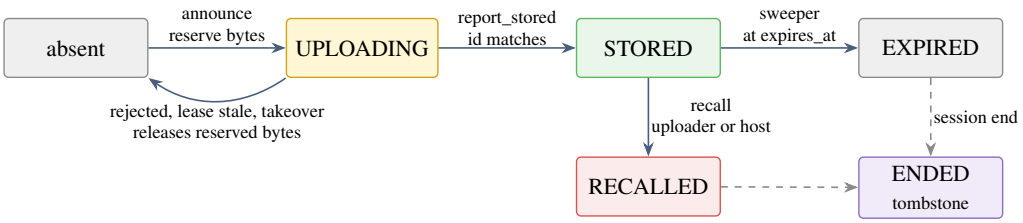


Figure 5: Photo status state machine (Redis)

Inside the image server, each upload has its own state (`RECEIVING`, `VERIFYING`, `VERIFIED`, `REPORTING`, then `STORED`, `REJECTED`, `SUPERSEDED` or `ENDED`), written to disk before every reply. A repeated `complete` after a timeout therefore returns the stored result instead of processing the photo again.

3.8 Algorithms

3.8.1 *Atomic Announce with Quota Reservation*

Announce decides, in one atomic step, whether the photo must be uploaded. Because Redis runs a function to completion before any other command, two members announcing the same photo at the same instant cannot both get `UPLOAD`, and concurrent announces cannot jointly exceed the quota.

Algorithm 1: announce (Redis function, one atomic step)

Input: session s, user u, hash h, size n, fresh id id.
Step 1: if s is not `ACTIVE` $\Rightarrow$ `410 SESSION_ENDED`; if u is not a member $\Rightarrow$ `403`; if u has 3 strikes $\Rightarrow$ `403`.
Step 2: if h is `STORED` or `EXPIRED` $\Rightarrow$ `ALREADY_UPLOADED`; if `RECALLED` $\Rightarrow$ `410`.
Step 3: if h is `UPLOADING` and its lease exists: same uploader $\Rightarrow$ `UPLOAD` with the *same* id and a fresh token; otherwise `IN_PROGRESS`. If the lease is gone and the reservation is older than 60 s, release it (*takeover*).
Step 4: count live reservations (releasing stale ones); if u has 3 or the session 20 $\Rightarrow$ `429 TOO_MANY_UPLOADS`.
Step 5: if $B_{\text{stored}} + B_{\text{reserved}} + n > C \Rightarrow$ `429 QUOTA_EXCEEDED`.
Step 6: `HSETNX` $h \rightarrow$ {`UPLOADING`, u, id}; $B_{\text{reserved}} \mathrel{+}= n$; create lease `upload:{s}:`$id$ with a 15-minute TTL; return `UPLOAD`, id.

The quota invariant that Step 5 maintains, for a session cap C (1 GB by default), is

$$B_{\text{stored}} + B_{\text{reserved}} \leq C, \tag{2}$$

where B_{reserved} moves to B_{stored} when the photo is stored and is released exactly once when the reservation ends in any other way (rejection, stale lease, takeover or session end). A job recomputes both counters every 5 minutes from the photos hash and corrects any drift.

3.8.2 *Fenced Report*

The image server's report is the only way a photo becomes `STORED`. It names the `upload_id`, which acts as a fencing token.

Algorithm 2: report_stored (Redis function)

Input: session s, hash h, upload id id, `stored_at`, `expires_at`.
Step 1: if s is not `ACTIVE` $\Rightarrow$ `410`; the image server deletes this upload's files.
Step 2: if the entry for h is missing or its id $\neq id \Rightarrow$ `409 UPLOAD_SUPERSEDED` (fenced out); files of id are deleted.
Step 3: if the entry is no longer `UPLOADING` $\Rightarrow$ `200 NOOP` (duplicate report).
Step 4: if the uploader has left $\Rightarrow$ release the reservation, `409`.
Step 5: mark `STORED`, move n bytes from reserved to stored, delete the lease, and `XADD` a `PHOTO_STORED` event, all in the same step; return `APPLIED`.

Because the event is appended in the same atomic step as the status change, a photo is `STORED` exactly when its event exists: there is no moment where one is visible without the other.

3.8.3 Resumable Chunked Upload

The upload works like the selective-repeat protocol of the transport layer, one level higher (Figure 6). Each chunk is an independent, idempotent `PUT` that carries its index and its CRC32C. The server stores chunks by index, so a repeated chunk overwrites itself. After a failure the client asks `GET /v1/uploads/{id}` which chunks arrived and sends only the others.

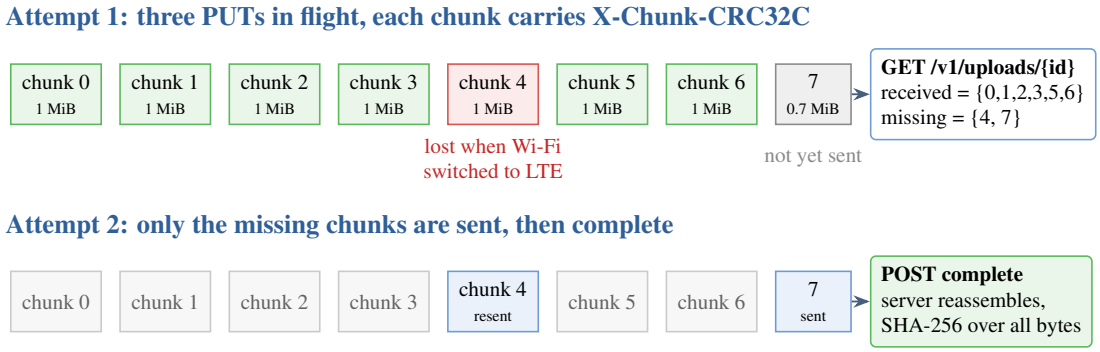


Figure 6: Resumable chunked upload: only lost chunks are sent again

The Android client sizes chunks from the throughput $\hat{B}$ it measured on earlier uploads, aiming at about two seconds of transfer per chunk:

$$c = \max\left(\text{clamp}\left(\hat{B}\cdot 2\text{ s},\ 256\text{ KiB},\ 8\text{ MiB}\right),\ \left\lceil \frac{S}{4096} \right\rceil\right), \tag{3}$$

rounded to 64 KiB, where S is the photo size. Without a measurement it uses the server's hint of 1 MiB. Larger chunks mean fewer requests on a fast link; smaller chunks lose less work when a slow link fails. Each chunk is retried up to 3 times with the backoff of Equation 1; after that the photo stays in a persistent queue and resumes when the network returns.

3.8.4 Gap-Free Event Delivery

A common way to build live updates is to replay missed events from a database and then subscribe to a live channel. Any event published between the end of the replay and the start of the subscription is lost (Figure 7, middle row). PHOTON avoids the gap by reading backlog and live events from the same source in one loop.

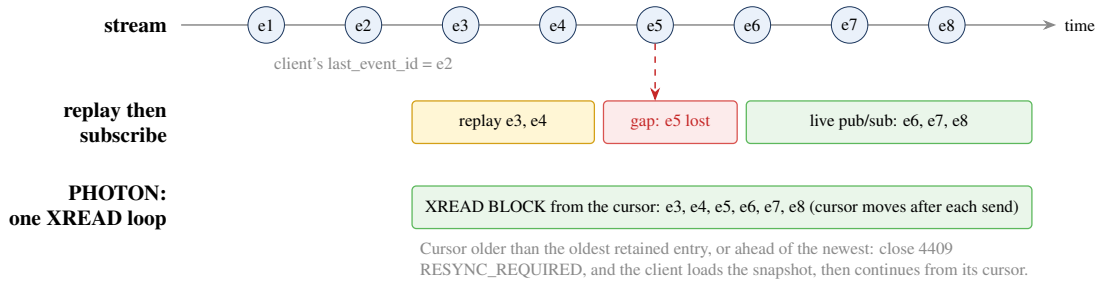


Figure 7: Replay-then-subscribe loses events; one XREAD loop from the cursor does not

Algorithm 3: event loop for one WebSocket

Step 1: read `Hello`; verify the JWT; check membership; let $c \leftarrow$ `last_event_id` (or the newest id for a new client).

Step 2: if c is older than the oldest retained entry, or newer than the newest $\Rightarrow$ close `4409 RESYNC_REQUIRED`.
Step 3: send `ready_cursor`; then loop: `XREAD BLOCK 15000 STREAMS session:{s}:events` c.
Step 4: for each entry (id, e): send e; $c \leftarrow id$. On `SESSION_ENDED` close `4410`.
Step 5: in parallel, answer pings, and close `4001` when the JWT expires; the client refreshes and reconnects from c.

On resync, and on every first join, the client loads the snapshot. The server reads the cursor *before* the state, so any photo stored after the cursor is also delivered as an event; the client simply drops the duplicate.

3.8.5 Client Deduplication: Exactly-Once Effect

Delivery is at-least-once: after a reconnect the same event may arrive again, and a photo may arrive both in a snapshot and as an event. The client makes the *effect* exactly-once with two checks:

$$\text{process}(e) \iff \text{id}(e) > c_{\max} \ \wedge \ \text{sha256}(e) \notin H_{\text{local}}, \tag{4}$$

where $c_{\max}$ is the highest event id already processed and H_{local} is the set of hashes already received or uploaded. Stream ids increase within a session, so the first check also discards anything older than the cursor.

3.8.6 Refresh-Token Rotation with a Grace Window

Refresh tokens are random, stored only as hashes, and rotated on every use. Reusing an old token normally means it was stolen, so the whole token family is revoked. Phones, however, legitimately send the same refresh twice when a response is lost. PHOTON allows a 30-second grace window: a token rotated less than 30 s ago that is presented again gets a new pair in the same family instead of revoking it. Outside the window, reuse revokes the family. The client also sends at most one refresh at a time (single-flight).

3.8.7 Retention with Two Sweepers

Retention has two separate guarantees (Figure 8). *Logical* expiry is exact: from `expires_at` no download succeeds, because both the orchestrator's status and the image server's own metadata are checked on every request. *Physical* deletion is bounded: two independent sweepers delete the file within 5 minutes, and an alert fires at 15 minutes. The image server's sweeper works only from its local metadata files, so even if Redis loses its state, files are still deleted on time.

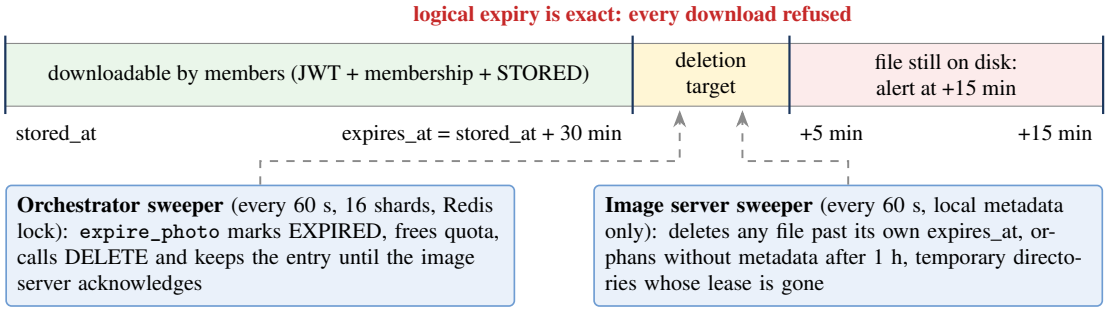


Figure 8: Retention: exact logical expiry and bounded physical deletion

3.9 Novelty and Innovation

The six ideas below are what distinguish PHOTON from existing photo-sharing tools. Each is described with its core idea, why it is new for this application, and how it works.

3.9.1 N1: Verify-Before-Announce with Content-Addressed Dedupe

Core Idea

A photo is identified by the SHA-256 of its bytes, uploaded at most once per session, and announced only after the image server has recomputed that hash itself. The client's hash is treated as a claim, never as a fact.

Why this is novel. Deduplication by client-supplied hash is a known optimisation, but it opens an attack: a member announces the hash of someone else's photo with different bytes, and every later upload of the real photo is answered "already uploaded". PHOTON makes the server the only judge of the hash and ties the announcement to that judgement, so dedupe and integrity come from the same check. A mismatch is rejected (`422 HASH_MISMATCH`), counts as a strike, and three strikes block the member's uploads.

How it works. The upload token names the claimed hash and size. The image server streams SHA-256 over the chunks while assembling them (1.8 GB/s on one core in our measurement), compares the result with the token, validates and decodes the image in a sandboxed subprocess, and only then sends the fenced `stored` report that makes the photo visible.

3.9.2 N2: Fencing Tokens on Every Cross-Service Message

Core Idea

Every message that changes upload state names its `upload_id`, and is applied only if that id is still the live reservation for the photo. A delayed message from a stalled uploader can never clear or revive a newer upload.

Why this is novel. Mobile uploads stall for minutes and resume later; servers restart; reports are retried. Without fencing, a late `rejected` from an abandoned attempt would delete a newer attempt's reservation, and a late `stored` could revive a cancelled one. PHOTON applies the fencing-token pattern from distributed locking to a mobile upload pipeline, together with leases and on-disk outboxes, so that every step is idempotent and resumable.

How it works. Figure 9 shows uploader A stalling, its lease expiring and uploader B taking over with a new id. When A's report finally arrives it carries `u1`, which no longer matches, so it is answered `409 UPLOAD_SUPERSEDED` and the image server deletes exactly A's files. All temporary paths contain the `upload_id`, so the cleanup cannot touch B's files.

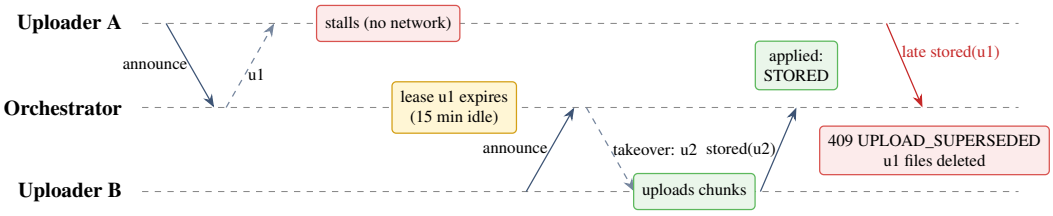


Figure 9: Fencing by `upload_id`: a late report from a stalled uploader is rejected

3.9.3 N3: One Read Loop for Backlog and Live Events

Core Idea

Each WebSocket reads the session's Redis Stream with `XREAD` starting after the client's cursor and keeps reading in the same loop. Backlog and live events come from one source, so there is no window in which an event can be missed.

Why this is novel. Most real-time designs combine a history query with a separate pub/sub subscription and must handle the race between them. Using the durable stream itself as the live channel removes the race. The stream is trimmed by age to the retention window plus one hour, and a client whose cursor is outside the retained range receives `RESYNC_REQUIRED`, so even a client that was offline for a day recovers completely from the snapshot. Push notifications are demoted to a hint: on any push, and on every app start, the client runs the same catch-up.

How it works. See Algorithm 3 and Figure 7. In the tests, 1,200 events were delivered exactly once and in order across random disconnections, and 3,060 of 3,060 events reached 153 sockets.

3.9.4 N4: A Transport-Aware Data Plane

Core Idea

Photo bytes travel over HTTP/3 (QUIC on UDP 443), fall back to HTTP/2 on TCP automatically through `Alt-Svc`, and are sent as small, checksummed, resumable chunks whose size adapts to the measured throughput.

Why this is novel. Consumer photo-sharing apps usually upload a photo in one request and restart it after a failure. PHOTON combines three transport-level techniques: QUIC's connection migration for network changes, per-chunk CRC32C with selective retransmission for loss, and throughput-based chunk sizing (Equation 3) for links from 2G to Wi-Fi. A failed transfer costs at most one chunk, not the whole photo.

How it works. Caddy terminates QUIC and TLS 1.3 at the edge; the Android app uses Cronet, which speaks HTTP/3 and falls back on its own. Each chunk carries `X-Chunk-CRC32C`; the server rejects a corrupted chunk and the client resends just that chunk.

3.9.5 N5: Ephemeral by Design, with Per-Request Authorisation

Core Idea

There are no shareable links. Every download is authorised again, on every request, by JWT, session membership, photo status and `expires_at`, and two independent sweepers delete each file within 5 minutes of expiry.

Why this is novel. Cloud albums keep photos until a user deletes them, and many sharing tools hand out bearer links that keep working once leaked. PHOTON treats the server copy as a transit buffer: logical expiry is exact, physical deletion is bounded and monitored, responses carry `Cache-Control: private, no-store` so no shared cache keeps a copy, and the image server refuses to serve a file past its own `expires_at` even if Redis state is stale.

How it works. See Figure 8. Recall and session end use the same deletion path with `expires_at` set to now.

3.9.6 N6: Background Capture Pipeline with Layered Echo Prevention

Core Idea

On Android, a foreground service and a content-URI WorkManager trigger notice new camera photos even when the app is in the background or its process was killed; a received photo saved to the gallery is recognised by five independent checks so it is never uploaded back.

Why this is novel. Saving a received photo fires the same gallery observer as taking one, and the operating system may re-encode the saved copy, so its hash differs. A naive app would upload every received photo back to the session, which would echo it to everyone again. PHOTON checks, in order: the asset id recorded when the photo was saved, the dedicated PHOTON folder, the owner package, a local hash set, and finally the server's session-wide dedupe.

How it works. On a real phone (Motorola edge 70 fusion, Android 16), a test photo taken with PHOTON in the background was stored in 2.1 s, 1.8 s after the app's process had been killed, and sync resumed by itself after an app update (Section 4.9.6).

4. Implementation

The server side, the reference client and all tooling are written in Python; the state transitions are a Lua library that runs inside Redis; the phone apps are native, because detecting new photos needs the operating system's media APIs. Everything described in this chapter is in the submitted source code and was run on the build machine; the Android app was also run on a real phone.

4.1 Technology Stack

Layer	Technology	Why this choice
Edge proxy	Caddy 2	TLS 1.3 with automatic certificates, HTTP/3 and `Alt-Svc` built in, reverse proxy with load balancing
Services	Python 3.12, FastAPI, uvicorn (uvloop, httptools)	One asynchronous event loop per process handles thousands of open sockets; HTTP and WebSocket in one framework
State and events	Redis 7.2, Redis Functions (Lua), Streams	Atomic multi-key transitions in one round trip; a durable, ordered, trimmable event log
Accounts	Postgres 16, SQLAlchemy 2 (async), asyncpg, Alembic	Durable, relational, migrations under version control
Wire format	Protocol Buffers 5	Compact binary bodies and WebSocket frames, one schema for Python, Kotlin and Swift
Tokens	PyJWT with Ed25519 (EdDSA)	Small, fast signatures; public keys published as a JWKS for rotation
Images	Pillow, pillow-heif in a sandboxed subprocess	JPEG, PNG, WebP and HEIC decoding with memory and CPU limits
Checksums	`hashlib` SHA-256, `google-crc32c`	Hardware-accelerated; 1.8 GB/s and 11.4 GB/s on one core
Push	firebase-admin (FCM), aioapns (APNs)	Wake-up hints for Android and iOS
Android	Kotlin, Jetpack Compose, WorkManager, Room, OkHttp, Cronet	Background work that survives process death; HTTP/3 through Cronet
iOS	Swift 6, PhotonKit package, SwiftUI	Shared core library, tested on Linux

Table 6: Technology stack

4.2 File Inventory

Directory	File	Purpose	LOC
orchestrator/lua/	photon.lua	All 24 state transitions as Redis Functions	556
orchestrator/routes/	events.py	WebSocket, snapshot, catch-up, close codes	228
	sessions.py, photos.py	Create, join, leave, kick, end; announce, recall, report	275
orchestrator/	svc.py, jobs.py, push.py	Function calls, sweepers and recount, FCM/APNs	453
imageserver/	pipeline.py	Chunks with CRC32C, assembly, SHA-256, outbox report	321
	routes.py, access.py	Upload and download routes, per-request authorisation	265
	outbox.py, store.py, sweeper.py	Durable reports, file layout and promotion, local sweeper	404
auth/	app.py, providers.py, tokens.py	OAuth, PKCE, Google ID tokens, GitHub, refresh rotation	565
common/	jwtutil.py, config.py, ...	JWT, settings, error contract, Redis pools, metrics	818
android/	65 Kotlin files	Observer, uploader, receiver, UI, tokens	7,172
ios/	PhotonKit and app sources	Upload engine, event sync, PKCE, receive pipeline	7,731
deploy/	docker-compose.yml, Caddyfile, ...	Edge, services, Redis with replica and ACLs, Prometheus	292
tests/	unit, integration, e2e, bench	Server tests and benchmarks	3,365
	All source files, including those not listed (plus 4,808 lines of app tests)		**24,356**

Table 7: Main source files and their size in lines of code

4.3 Server Implementation

One library of Redis Functions. Every state transition is a function in one Lua library, loaded with `FUNCTION LOAD` and called with `FCALL`: `create_session`, `join`, `leave`, `kick`, `end_session`, `announce`, `report_stored`, `report_rejected`, `recall`, `expire_photo`, `recount`, `touch_lease`, and read-only checks such as `upload_check`, `download_check`, `snapshot` and `events_since`. Redis runs each function to completion before serving any other command, so every check and every write of one transition happen as one atomic step on the server, in one network round trip. Unlike loose `EVAL` scripts, functions survive a restart and replicate to the replica. Each function declares only keys of one session, which share the `{sid}` hash slot, so the library can later move to Redis Cluster unchanged.

Three services. Auth, the orchestrator and the image server are separate FastAPI applications with their own Redis ACL user. The orchestrator can run any number of replicas behind the proxy without sticky routing, because a WebSocket reads its events from the Redis stream, not from the process that handled the upload.

Connection pools sized for blocking reads. Each open WebSocket holds one Redis connection in `XREAD BLOCK`. These connections come from a separate blocking pool without a socket timeout (200 per replica by default); the pool for ordinary commands has 500 connections, and callers wait up to 5 s for a free one instead of failing. Both settings came from measurements (Section 4.10).

Durability on the image server. The image server writes each upload's state to disk before it replies, and writes a pending-report record before it reports to the orchestrator. A crash between storing a file and reporting it is recovered at start-up from that record. Files are promoted to their final path `photos/{sid}/{sha256}/` only after the orchestrator answers 200, with an atomic rename and `fsync`.

4.4 Network Edge and Deployment

Figure 10 shows the deployment on one host with Docker Compose. Only Caddy publishes ports: 443/tcp for HTTPS and WebSocket, 443/udp for QUIC, and 80/tcp for certificate challenges and redirects. Every service and store sits on a private Docker network. Redis requires TLS and has one ACL user per service: the image server, for example, can read `session:*` and refresh upload leases, and nothing else.

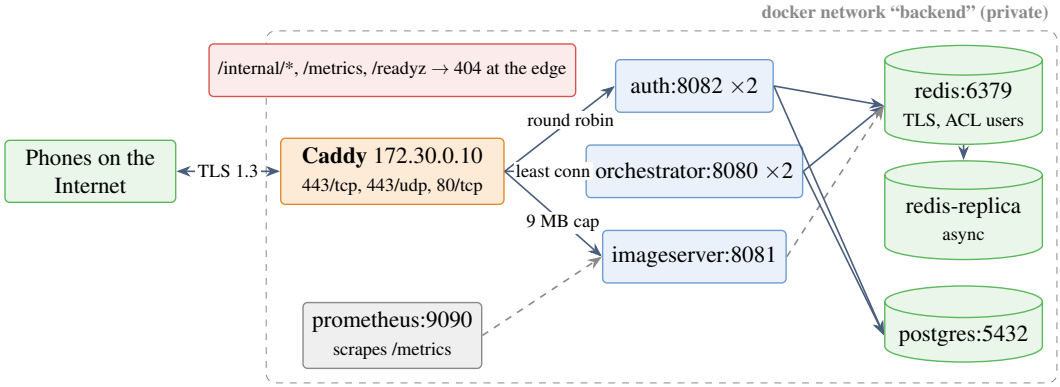


Figure 10: Deployment and network topology

The edge configuration fits on one page. The global `protocols h1 h2 h3` line enables HTTP/3; Caddy then sends `Alt-Svc: h3=":443"` on HTTP/2 responses, and clients that can use UDP switch over.

```
{ servers { protocols h1 h2 h3 } }            # HTTP/3 (QUIC) with HTTP/2 fallback
{$PHOTON_PUBLIC_HOST} {
    @private path /internal/* /metrics /readyz
    respond @private 404                        # never reachable from outside
    @auth path /v1/auth/* /v1/devices/* /.well-known/*
    handle @auth { reverse_proxy auth:8082 { lb_policy round_robin } }
    handle { reverse_proxy orchestrator:8080 { lb_policy least_conn
                                               stream_timeout 24h } }   # WebSockets
}
{$PHOTON_IMAGE_HOST} {
    @chunks path /v1/uploads/*/chunks/*
    request_body @chunks { max_size 9MB }       # largest chunk is 8 MiB
    reverse_proxy imageserver:8081 { flush_interval -1 }
}
```

The services trust `X-Forwarded-For` only when the direct peer is a configured proxy, so a client cannot forge its address to escape per-IP rate limits. uvicorn's keep-alive timeout (130 s) is longer than Caddy's upstream keep-alive (2 minutes), so the proxy never reuses a connection the service has just closed.

4.5 Authentication and Security

Figure 11 shows the browser sign-in. The `state` value is stored in Redis for 10 minutes and read with `GETDEL`, so it works exactly once; the PKCE verifier proves that the app which started the login is the one finishing it; redirect URIs are matched exactly. On Android, Google sign-in uses the system account picker instead: the app gets a single-use nonce from the server, and the server checks the ID token's issuer, audience, expiry and nonce.

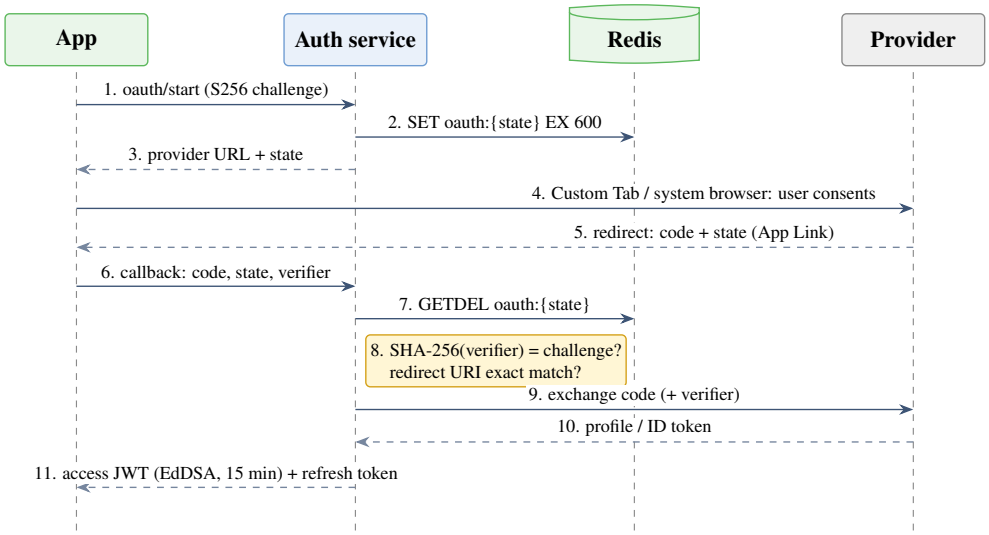


Figure 11: Browser sign-in with OAuth 2.0 Authorization Code and PKCE

Threat	Mitigation
Eavesdropping or tampering on any hop	TLS 1.3 everywhere, including Redis and service-to-service calls; QUIC encrypts its transport headers too
Stolen authorization code	PKCE: the code is useless without the verifier, which never leaves the phone
Replayed or forged login	Single-use `state` (`GETDEL`); single-use nonce for Google ID tokens; exact redirect URIs
Stolen refresh token	Rotation on every use; reuse outside the 30 s grace window revokes the whole family
Leaked access token	15-minute lifetime; WebSocket closed with 4001 at expiry; token sent in the first frame, never in a URL
Forged hash to poison dedupe	Server recomputes SHA-256; mismatch is a strike; 3 strikes block uploads
Malicious image file (decoder exploit, decompression bomb)	Type allow-list, magic bytes, 25 MB and 100 MP limits, decoding in a separate process with memory, CPU and wall-clock limits
Leaked event or link	Events carry no URL; every download checks JWT, membership, status and expiry; `no-store`
Abuse as free file hosting	1 GB per session, 3 live uploads per user and 20 per session, 20 announces a minute, 30-minute retention

Table 8: Threats and how PHOTON handles them

4.6 Android Client

The Android app is where the networking meets the limits of a mobile operating system (Figure 12). Android stops ordinary background services within minutes, so while a session is active the app runs a foreground service with a visible notification, whose `ContentObserver` only records new photos in a local Room table and enqueues work. A second path, a WorkManager job triggered by a content URI, fires even when the app's process has been killed. WorkManager owns every slow step: hashing, removing GPS data, announcing, uploading with retries, and resuming after the network returns.

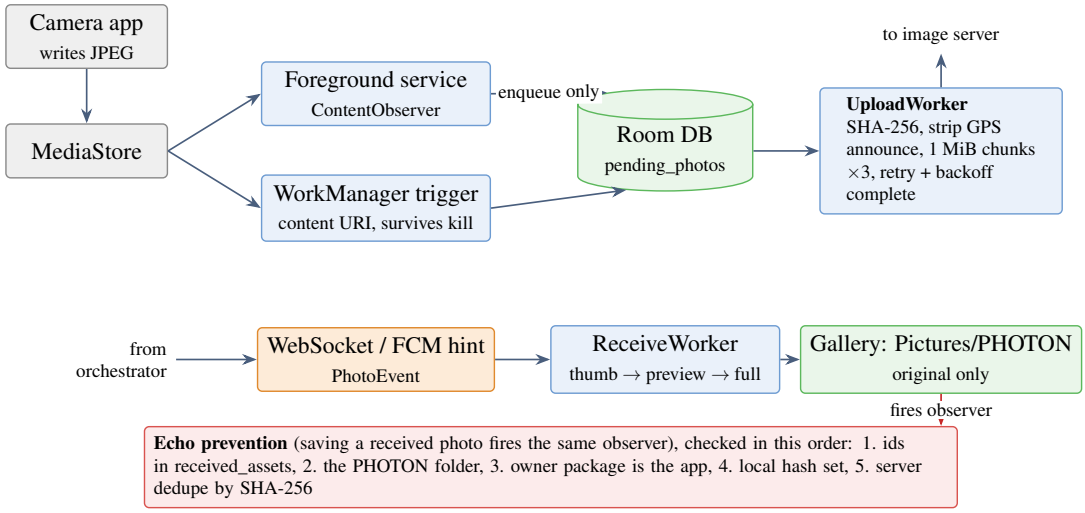


Figure 12: Android client: capture, upload, receive and echo prevention

On the receiving side, the app shows the thumbnail as soon as the event arrives, then the preview, and fetches the original on Wi-Fi or while charging if the user chose that. Only the original is saved to the gallery, in a `Pictures/PHOTON` folder, with its original capture time so it lands in the right place in the timeline. A reconcile scan on every start, restart and boot catches anything the observer missed. The app also restarts its service after an app update (`MY_PACKAGE_REPLACED`) and asks the user to exempt it from battery optimisation.

4.7 iOS Client

iOS does not let an app run continuously in the background, so the iOS design states this openly: photos taken while the app is suspended are uploaded on its next launch or return to the foreground, and a silent APNs push may wake it briefly. The shared logic is a Swift package, PhotonKit, with the upload engine, chunk planner, PKCE, token manager, event sync and receive pipeline. It builds and its 90 tests pass on Linux with Swift 6.3; the SwiftUI app target has not been compiled, because that needs Xcode on macOS.

4.8 Testing

The tests start a real Redis, an embedded Postgres and all three services on random ports. Time is driven through a shared clock module, so expiry and grace windows are tested without waiting. Table 9 lists the suites.

Suite	What it proves
Unit	Error contract, ids, one Redis slot per function call, token verification, the decoder against hostile files, push routing
`test_auth`, sign-in	Single-use `state`, PKCE, exact redirect URIs, refresh grace window and reuse revocation, no merging by e-mail, Google nonce, GitHub flow
`test_upload`	Resumable chunks, idempotent `complete` in every state, hash mismatch, crash recovery through the outbox
`test_fencing`	One upload per hash, quota races, stale reports fenced out, Redis-loss recovery, recall, strikes
`test_model`	Random sequences of every transition against the real Lua functions (hypothesis), checking the quota and event invariants after each step
`test_events`	1,200 events delivered once and in order across random reconnects; resync; close codes; downloads with `Range`
`test_retention`	Exact logical expiry, both sweepers, retried deletes, orphans, Redis loss
Result	**155 server tests pass** (67 s); 135 Android unit tests and 90 PhotonKit tests pass; `ruff` clean; `mypy --strict` clean

Table 9: Test suites and what they prove

4.9 Performance Evaluation

4.9.1 Setup

The benchmark suite (`scripts/bench.sh`) ran on one laptop: Intel Core i5-13450HX (16 threads), 15 GB RAM, Python 3.12.3, Redis 7.2.4. Redis, Postgres, all services and the load generator share the machine and, for the HTTP tests, one Python event loop. The numbers are therefore relative measurements of this build over loopback, not production capacity.

4.9.2 State Transitions and Primitives

Operation	p50	p95	p99
`announce` (new photo), 1,000 calls	0.08 ms	0.17 ms	0.26 ms
`report_stored` with event append	0.07 ms	0.17 ms	0.24 ms
`download_check` (every download)	0.07 ms	0.15 ms	0.28 ms
`events_since` (500 events)	14.4 ms	15.3 ms	32.1 ms
`snapshot` (1,000 stored photos)	10.9 ms	14.6 ms	18.8 ms
Throughput with 64 calls in flight: 14,038 `download_check`/s. Ed25519: sign 34,230/s, verify 9,842/s.			
SHA-256: 1,831 MB/s; CRC32C: 11,403 MB/s (one core).			

Table 10: Latency of Redis functions and cost of tokens and checksums

A state transition costs under a tenth of a millisecond inside Redis, so the atomic design does not limit throughput; the HTTP layer does.

4.9.3 Announce over HTTP and Little's Law

Figure 13 shows the announce endpoint (JWT check, protobuf decoding, one Lua call and signing an upload token) under 1, 16 and 64 concurrent clients. Throughput rises from 545 to about 700 requests per second and then stays flat: the single service process is saturated. Beyond saturation, extra clients only wait in the queue. Little's law relates the number of requests in the system L, the throughput λ and the time each request spends in the system W:

$$L = \lambda W \quad \Rightarrow \quad W = \frac{L}{\lambda} = \frac{64}{704\ \mathrm{s}^{-1}} \approx 91\ \mathrm{ms}. \tag{5}$$

The measured mean is 75 ms. The remaining 16 ms is time each request spends inside the load generator before it is timed, because the generator shares the event loop with the services. At 16 clients the prediction is 23 ms against 19 ms measured. The model explains the result: latency grows linearly with concurrency once the server is saturated, so capacity is added with more orchestrator replicas, which need no coordination.

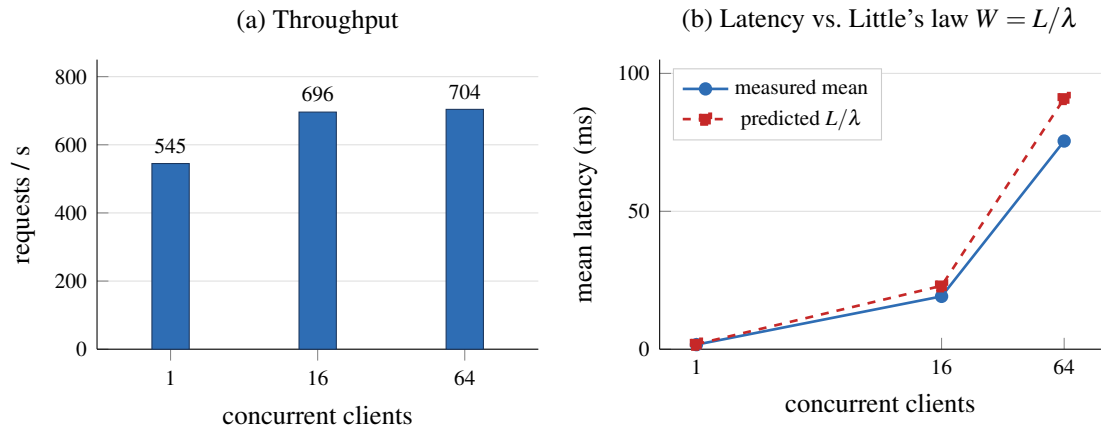


Figure 13: Announce over HTTP: throughput saturates at about 700 requests/s; latency follows Little's law

4.9.4 Upload Path

Figure 14 breaks the end-to-end upload time into its three phases for four photo sizes, with 1 MiB chunks sent three at a time. Over loopback the chunk transfer is small; almost all the time is `complete`, which assembles the file, hashes it, validates and decodes it in a subprocess, makes the thumbnail, writes the metadata and sends the fenced report. On a phone the chunk phase is bounded by the radio link instead.

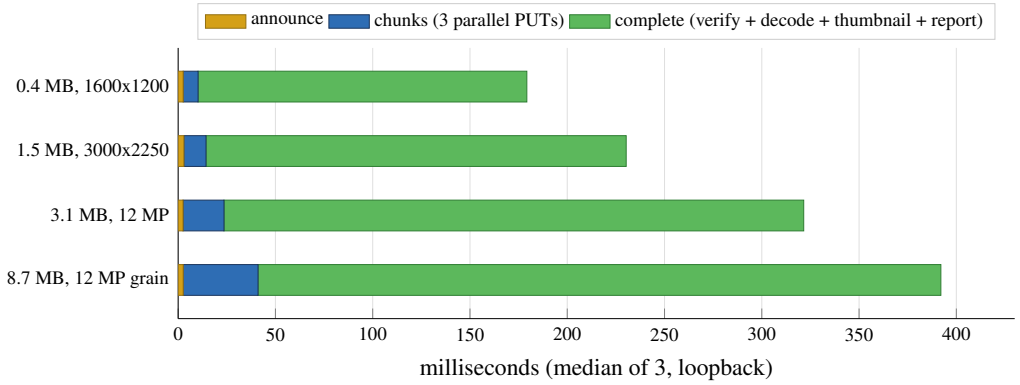


Figure 14: Where upload time goes, by photo size (median of 3)

Decoding dominates: even a tiny JPEG costs about 48 ms, the price of starting a fresh, resource-limited process for every decode, so that a malicious file can exhaust only its own worker. A 12 MP JPEG costs 97 ms to validate and 151 ms for the thumbnail; a 12 MP HEIC costs about six times as much.

4.9.5 Event Fan-out and Throughput

Measurement	p50	p95	p99
Event appended → frame received, 10-member session (200 deliveries)	2.6 ms	4.3 ms	4.5 ms
Upload start → last of 10 members received (20 photos)	159 ms	167 ms	168 ms
Event appended → frame received, 153 sockets in 17 sessions (3,060 deliveries)	28.8 ms	31.3 ms	35.8 ms
8 members uploading at once: 48 photos (45 MB) in 1.94 s = 24.8 photos/s, 23.2 MB/s, all stored.			
Load test, 20 sessions × 5 members: 300 of 300 photos stored, 1,200 of 1,200 deliveries.			

Table 11: Event delivery and upload throughput

A photo reaches all ten members of a session about 160 ms after its upload starts, and the fan-out itself (stream to socket) takes under 5 ms. With 153 sockets, every one of the 3,060 events was delivered.

4.9.6 Real Device Results

The Android app was tested on a Motorola edge 70 fusion with Android 16, connected to the development server over USB (`adb reverse`) and over Wi-Fi.

Scenario	Result
Photo taken with the camera app, PHOTON in the background	Stored 14 s after capture
Test photo written to the gallery, PHOTON in the background	Stored 2.1 s after it was written
Same, after the app's process was killed	Stored 1.8 s after (service restarted, trigger fired)
App updated during a session	Sync resumed without opening the app

Table 12: Results on a real phone

The 14 s for a real camera photo includes the time the camera app takes to finish writing the file and publish it to MediaStore, which PHOTON cannot see until it happens.

4.10 Problems Found by Measurement

Running the system on a phone and under load found faults that the functional tests had not. Each was fixed and is now covered by a test.

Symptom	Cause	Fix
Idle WebSockets on the phone closed with 1011 every few seconds	The Redis client's default 5 s socket timeout cut every `XREAD BLOCK 15000`	Blocking pool without a socket timeout, with TCP keepalive
150 sockets connecting at once: 50 closed	redis-py's default pool fails the 101st concurrent command	Blocking pools sized by configuration (500 and 200); callers wait up to 5 s
Phone on Wi-Fi could not reach the server	The app was built for `127.0.0.1`, which on a phone is the phone itself	Install script builds for the laptop's LAN address (Figure 19)
Photos uploaded only while the app was open	Background service stopped; no trigger after process death	Foreground service plus content-URI trigger plus restart after update

Table 13: Faults found by the device test and the benchmarks

4.11 Screenshots of the System

The screenshots below come from a real run of the development stack (`photon-dev`) with two users of the command-line client, Alice and Bob, on one machine.

alice and bob: sign in, create, join, upload (photon-cli)

```
$ photon-cli --profile alice login
logged in as 07d2d064-e083-4be2-9704-12993284eb0f

$ photon-cli --profile bob login
logged in as fee37c28-7aa9-4938-8912-d2e116f0702b

$ photon-cli --profile alice create
session  JzTJvmyAkb--B98cmtQ64w
role     host
code     1H17G5T9
invite   https://photon.example/j/yyjP8QY3ccmaWnZubomrmA
members  07d2d064-e083-4be2-9704-12993284eb0f

$ photon-cli --profile bob join 1H17G5T9
session  JzTJvmyAkb--B98cmtQ64w
role     member
code     1H17G5T9
invite   https://photon.example/j/yyjP8QY3ccmaWnZubomrmA
members  07d2d064-e083-4be2-9704-12993284eb0f, fee37c28-7aa9-4938-8912-d2e116f0702b

$ photon-cli --profile alice upload pics/IMG_2041.jpg pics/IMG_2042.jpg
IMG_2041.jpg: STORED b0423e8f021f3aa8
IMG_2042.jpg: STORED a6e1d483ff4eb9a6

$ photon-cli --profile alice upload pics/IMG_2041.jpg
IMG_2041.jpg: ALREADY_UPLOADED b0423e8f021f3aa8
```

Figure 15: Alice and Bob sign in, Alice creates a session, Bob joins with the code, Alice uploads two photos, and a repeated upload is answered `ALREADY_UPLOADED`

bob: live events over the WebSocket, snapshot, byte-exact copies

```
$ photon-cli --profile bob listen --save-dir ./received
1793623947107-0 PHOTO_STORED b0423e8f021f3aa8 by 07d2d064-e083-4be2-9704-12993284eb0f (876394 B)
1793623947708-0 PHOTO_STORED a6e1d483ff4eb9a6 by 07d2d064-e083-4be2-9704-12993284eb0f (880149 B)

$ photon-cli --profile bob snapshot
cursor 1793623947708-0
members 07d2d064-e083-4be2-9704-12993284eb0f, fee37c28-7aa9-4938-8912-d2e116f0702b
(snapshot) PHOTO_STORED a6e1d483ff4eb9a6 by 07d2d064-e083-4be2-9704-12993284eb0f (880149 B)
(snapshot) PHOTO_STORED b0423e8f021f3aa8 by 07d2d064-e083-4be2-9704-12993284eb0f (876394 B)

$ sha256sum pics/*.jpg received/*
b0423e8f021f3aa888f93b8b7b485e5bfd1120e108624cef76b2e0650b0fc2aa  pics/IMG_2041.jpg
a6e1d483ff4eb9a61a35cd86e66820568c98c173b85d537d6322fbb746974df4  pics/IMG_2042.jpg
a6e1d483ff4eb9a61a35cd86e66820568c98c173b85d537d6322fbb746974df4  received/a6e1d483ff4eb9a6.jpg
b0423e8f021f3aa888f93b8b7b485e5bfd1120e108624cef76b2e0650b0fc2aa  received/b0423e8f021f3aa8.jpg
```

Figure 16: Bob receives both `PHOTO_STORED` events live over the WebSocket; the snapshot agrees; the received files are byte-identical to the originals (same SHA-256)

```
recall and end: downloads refused, events delivered

$ photon-cli --profile alice recall a6e1d483ff4eb9a61a35cd86e66820568c98c173b85d537d6322fbb746974df4

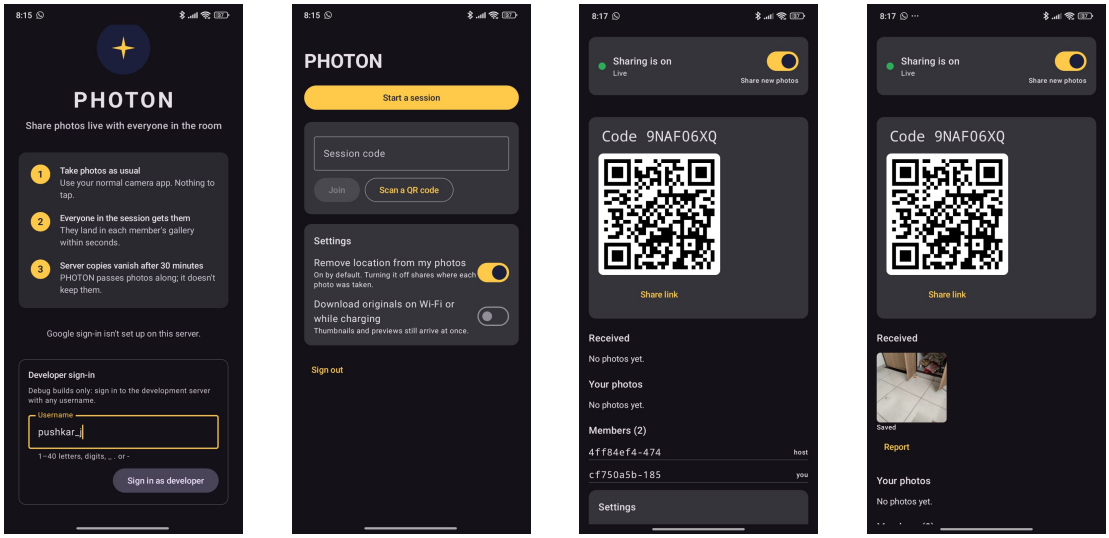
$ photon-cli --profile bob download a6e1d483ff4eb9a61a35cd86e66820568c98c173b85d537d6322fbb746974df4
  -o /dev/null
error: 410 PHOTO_RECALLED: Photo recalled

$ photon-cli --profile alice end

$ photon-cli --profile bob listen        # running during the recall and the end
(snapshot) PHOTO_STORED a6e1d483ff4eb9a6 by 07d2d064-e083-4be2-9704-12993284eb0f (880149 B)
(snapshot) PHOTO_STORED b0423e8f021f3aa8 by 07d2d064-e083-4be2-9704-12993284eb0f (876394 B)
1793623961975-0 PHOTO_RECALLED a6e1d483ff4eb9a6 by 07d2d064-e083-4be2-9704-12993284eb0f (0 B)
1793623962794-0 SESSION_ENDED
stopped: SESSION_ENDED
```

Figure 17: Alice recalls a photo: Bob's download is refused with `410 PHOTO_RECALLED` and his listener receives `PHOTO_RECALLED`, then `SESSION_ENDED` when Alice ends the session

Figure 18 shows the Android app in a session with two members: developer sign-in, offered because the development server has no Google client configured (a); starting or joining a session (b); the session code, its QR and the members (c); and a photo from the other member, received and saved to the gallery (d).



(a) Sign-in (b) Start or join (c) Live session (d) Photo received

Figure 18: The Android app: sign-in, the home screen, a live session with two members, and the same session after a photo from the other member was received and saved

4.12 Installation Guide

4.12.1 Run the Server and the Tests

The server needs Python 3.12 and a C compiler (Redis 7.2 is built from source); Postgres comes from a Python wheel, so no Docker is needed for development. The Android app needs JDK 17 and the Android SDK.

Quick Start

```
scripts/dev-deps.sh                          # builds Redis 7.2 into .devtools/
python3 -m venv .venv && .venv/bin/pip install -e ".[dev]"
.venv/bin/pytest -q                          # 155 tests
.venv/bin/photon-dev                         # API on :8080, image server on :8081
```

```
scripts/bench.sh                              # benchmarks -> tests/bench/results/latest.md
```

4.12.2 Run on an Android Phone

A phone reaches the laptop in one of two ways (Figure 19). Over USB, `adb reverse` forwards the phone's `127.0.0.1:8080` through the cable to the laptop, so the app needs no network configuration. Over Wi-Fi, the server listens on all interfaces and the app is built with the laptop's LAN address; both must be on the same subnet.

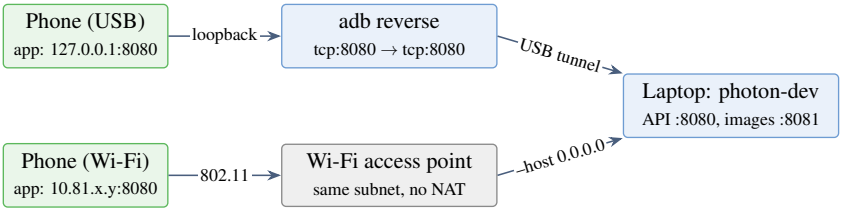


Figure 19: Two network paths from a test phone to the development server

Phone Setup

```
scripts/phone-backend.sh                    # terminal 1: backend reachable from phones
scripts/phone-install.sh                    # terminal 2: USB: adb reverse, build, install
    , open
scripts/phone-install.sh --lan 10.81.x.y    # or Wi-Fi: build for the laptop's address
scripts/phone-diagnose.sh                   # if photos stop arriving
```

4.12.3 Deploy

Production Deployment

```
python3 tools/gen_keys.py deploy/keys          # Ed25519 keys, one per service
scripts/make-certs.sh deploy/certs             # internal CA for Redis TLS
cp deploy/users.acl.example deploy/users.acl   # set Redis passwords
cp deploy/.env.example deploy/.env             # host names, OAuth clients
docker compose -f deploy/docker-compose.yml up -d --build
```

4.13 Comparison with Existing Systems

Table 14 compares PHOTON with the tools people use today to exchange event photos. The comparison is based on the publicly documented, default behaviour of each product; "Partial" means the feature exists only in a limited form. Figure 20 totals the table, counting a tick as 1 and Partial as 0.5.

Feature	PHOTON	Google Photos	iCloud SPL	AirDrop	Quick Share	WhatsApp group
Shares new camera photos with the group automatically	✓	Partial	✓	✗	✗	✗
Android and iOS members in one group	✓	✓	✗	✗	✗	✓
Members need not be near each other	✓	✓	✓	✗	✗	✓
No long-lived copy on a server	✓	✗	✗	✓	✓	Partial
Original quality by default	✓	Partial	✓	✓	✓	✗
Thumbnail first, original later	✓	✓	✓	✗	✗	✓
Uploader can recall a shared photo	✓	✓	✓	✗	✗	✓
Time-boxed group joined by code or QR	✓	✗	✗	✗	✗	✗
Open source and self-hostable	✓	✗	✗	✗	✗	✗
Score / 9	**9**	**5**	**5**	**2**	**2**	**4.5**

Table 14: Feature comparison: PHOTON and existing photo-sharing tools

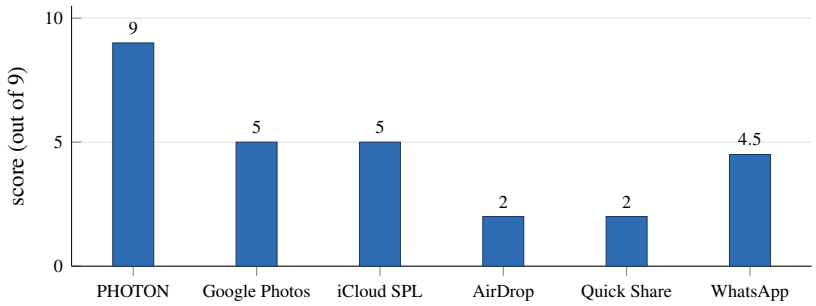


Figure 20: Feature score of PHOTON and existing tools (from Table 14)

The table counts only the features PHOTON was designed for. The existing tools are ahead elsewhere: WhatsApp encrypts end to end, while PHOTON's server can read a photo during its 30 minutes; Google Photos and iCloud keep long-term albums and handle video; AirDrop and Quick Share work peer to peer without any server; Apple's own library syncs in the background on iOS; and all of them are tested at a scale far beyond one machine and one phone.

5. Conclusion

5.1 Summary of Achievements

PHOTON turns a group of phones into a live, shared camera roll. It solves the problem stated in Section 1 with standard protocols, used carefully:

- **Verified before announced.** No member is told about a photo until the server has recomputed its SHA-256 and stored it, so content-addressed dedupe cannot be poisoned by a false claim.
- **Reliable over unreliable networks.** Uploads are cut into checksummed chunks sent over HTTP/3 with HTTP/2 fallback; only lost chunks are resent, and every request is idempotent, so a retry after any failure is safe.
- **No gaps, no duplicates.** One `XREAD` loop per socket delivers backlog and live events from the same durable stream; client-side deduplication by event id and hash turns at-least-once delivery into an exactly-once effect. Tests delivered 1,200 events across random reconnects, and 3,060 of 3,060 to 153 sockets.
- **Safe under concurrency and delay.** All state transitions are atomic Redis functions, and every cross-service report is fenced by its `upload_id`, so a stalled client can never undo newer state.

- **Ephemeral by design.** Logical expiry is exact at 30 minutes, physical deletion is bounded by two independent sweepers, and there are no shareable links.
- **Working on a real phone.** The Android app synced photos within about 2 s in the background, also after its process was killed and after an update; 155 server tests, 135 Android tests and 90 iOS-core tests pass.

5.2 Limitations

1. **No end-to-end encryption.** The server can read photos during their 30 minutes. Encrypting each photo with a per-session key held only by members would remove this, at the cost of server-side thumbnails and validation.
2. **HTTP/3 not measured on a phone.** HTTP/3 is configured at the edge and supported by the Android client, but the device tests ran against the development server over HTTP/1.1, without the edge proxy.
3. **iOS app not compiled.** The PhotonKit core is built and tested; the SwiftUI app needs Xcode.
4. **Single machine.** Benchmarks and the load test ran on one laptop, and the Docker deployment was written but not run on the build machine.

5.3 Future Work

1. **End-to-end encryption** with a session key exchanged through the join flow, and thumbnails made on the uploading phone.
2. **Measuring HTTP/3 against HTTP/2** on real mobile links, including uploads that continue across a Wi-Fi to LTE handover through QUIC connection migration.
3. **Peer-to-peer delivery** between phones on the same Wi-Fi network (for example with WebRTC data channels), using the server only for signalling and verification.
4. **Object storage and Redis Cluster** for horizontal scaling; the keys already carry `{sid}` hash tags and the storage layout is ready for S3.
5. **Video and Live Photos**, with resumable uploads of larger files and streaming previews.

5.4 Final Remarks

The project showed that the hard part of a "simple" sharing feature is the network between the phones: lost packets, changing addresses, stalled uploads, duplicated retries and missed notifications. Each was handled with a standard mechanism from computer networks, namely QUIC, selective retransmission, idempotent requests, leases, fencing tokens and a durable event log, and each was tested both in code and on a real device.

6. Bibliography

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[2] R. Fielding, M. Nottingham and J. Reschke, “HTTP Semantics,” RFC 9110, IETF, Jun. 2022.

[3] M. Thomson and C. Benfield, “HTTP/2,” RFC 9113, IETF, Jun. 2022.

[4] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, IETF, May 2021.

[5] M. Bishop, “HTTP/3,” RFC 9114, IETF, Jun. 2022.

[6] M. Nottingham, P. McManus and J. Reschke, “HTTP Alternative Services,” RFC 7838, IETF, Apr. 2016.

[7] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, Aug. 2018.

[8] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, Dec. 2011.

[9] D. Hardt, “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012.

[10] N. Sakimura, J. Bradley and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, IETF, Sep. 2015.

[11] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015.

[12] I. Liusvaara, “CFRG Elliptic Curve Diffie-Hellman (ECDH) and Signatures in JSON Object Signing and Encryption (JOSE),” RFC 8037, IETF, Jan. 2017.

[13] A. Petersson and M. Nilsson, “Forwarded HTTP Extension,” RFC 7239, IETF, Jun. 2014.

[14] J. Satran *et al.*, “Internet Small Computer Systems Interface (iSCSI),” RFC 3720, IETF, Apr. 2004 (CRC32C, Appendix B.4).

[15] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, Aug. 2015.

[16] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017, ch. 8 (fencing tokens).

[17] Redis Ltd., “Redis Streams” and “Redis Functions,” Redis documentation. Available: https://redis.io/docs/latest/develop/data-types/streams/

[18] Google, “Protocol Buffers Language Guide (proto3).” Available: https://protobuf.dev/programming-guides/proto3/

[19] Google, “Firebase Cloud Messaging.” Available: https://firebase.google.com/docs/cloud-messaging

[20] Apple Inc., “Sending notification requests to APNs.” Available: https://developer.apple.com/documentation/usernotifications

[21] Android Developers, “Android Debug Bridge (adb).” Available: https://developer.android.com/tools/adb

[22] Android Developers, “Schedule tasks with WorkManager” and “Foreground services.” Available: https://developer.android.com/develop/background-work

[23] Caddy Web Server documentation, “Global options: protocols.” Available: https://caddyserver.com/docs/caddyfile/options